

# Language Learning Platform

Guilherme Junio da Silva

ID: 23413271

Project (HDCSDEV\_INT)

## Interim Progress report

Date (06/28/2025):

## **1. Project Overview**

### **1.1 Background**

Learning a new language can be challenging, especially when it comes to remembering vocabulary and using it effectively in context. Many students rely on flashcards and repeated exposure to practice, but traditional tools often fail to adapt to individual learning needs. Generic flashcard apps typically present the same content in the same way to every user, without considering which words or phrases are harder for them.

This project also draws on my personal experience as an English teacher. One of the main challenges I faced was encouraging students to practice the language consistently outside the classroom. Many of them struggled to stay engaged or did not know how to review content effectively on their own. A tool like this – combining structured quizzes with adaptive logic – would have been very useful to support their independent study and reinforce what was taught in class.

Now, as a student in computing with an interest in language learning and educational technology, I see an opportunity to create a platform that adapts to each learner's performance and helps them focus on what they find most difficult. This project builds on previous work that I have done with flashcard-based learning tools and expands it into a more complete and interactive system.

### **1.2 Objective**

The main goal of this project is to develop a web-based language learning platform where users can log in, complete vocabulary quizzes, and view their progress over time. The system will adapt to their performance by showing difficult flashcards more frequently and reducing repetition of known items – using the spaced repetition technique to reinforce memory effectively.

Users will be able to select the quiz difficulty, track their quiz accuracy, and get feedback on which areas they need to improve. The application will aim to deliver a simple but intelligent study experience, supporting learners who prefer to study independently. The platform will be responsive and user-friendly, with a focus on structure, clarity, and meaningful analytics.

## 2. Project Expected Deliverables

### 2.1 Documents

The following documents will be submitted as part of the project lifecycle.

| Document                          | Description                                                                                                                             | Date Due               | Submitted |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------|-----------|
| Project Proposal                  | A document that outlines the project idea                                                                                               | 7 <sup>th</sup> June   | Yes       |
| Project Requirement Specification | A detailed breakdown of the system features, user interfaces and system design.                                                         | 28 <sup>th</sup> June  | Yes       |
| Interim Project Report            | A document that outlines the progress made so far, the planning process, and expected outcomes.                                         | 28 <sup>th</sup> June  | Yes       |
| Final Report                      | To be completed near the end of the project. This will describe the final implementation, testing results and evaluation of the system. | 8 <sup>th</sup> August | No        |
| Final Presentation Video          | A video presentation showcasing the completed system and its core functionalities.                                                      | 8 <sup>th</sup> August | No        |
| Source Code                       | The application code, submitted through GitHub or a Zip file.                                                                           | 8 <sup>th</sup> August | No        |

### 2.2 Application Functionality

The core functionalities of the language learning platform are as follows:

**User Authentication:** Students can register and log in to access their personalized content.

**Flashcard Management:** Flashcards are stored in private decks and used to generate quiz questions. Each flashcard contains a word or sentence in the target language and its translation.

**Quiz Mode:** Students can take short quizzes generated from flashcards. The system allows the users to select the difficulty level of the quiz.

**Spaced Repetition Logic:** The platform adjusts how frequently flashcards appear based on student performance. Cards that are marked as difficult appear more often, while easier ones are shown less frequently.

**Progress Tracking:** Users can view their quiz history, success rate, and areas that need improvement through a dashboard.

**Responsive Design:** The application will be accessed from both desktop and mobile devices, providing a user-friendly experience.

### 3. Project Targets Met So Far

#### 3.1 Documents Completed

**Project Proposal:** The proposal has been completed and submitted. It outlines the main objectives of the application, the proposed system structure, and a development plan using Trello.

**Trello Board:** A project board has been created to organize and track tasks. It contains columns for Backlog, To Do, In Progress, Testing and Done. Tasks are labeled by week and category (e.g., frontend, backend, documentation) to support time management and prioritization.

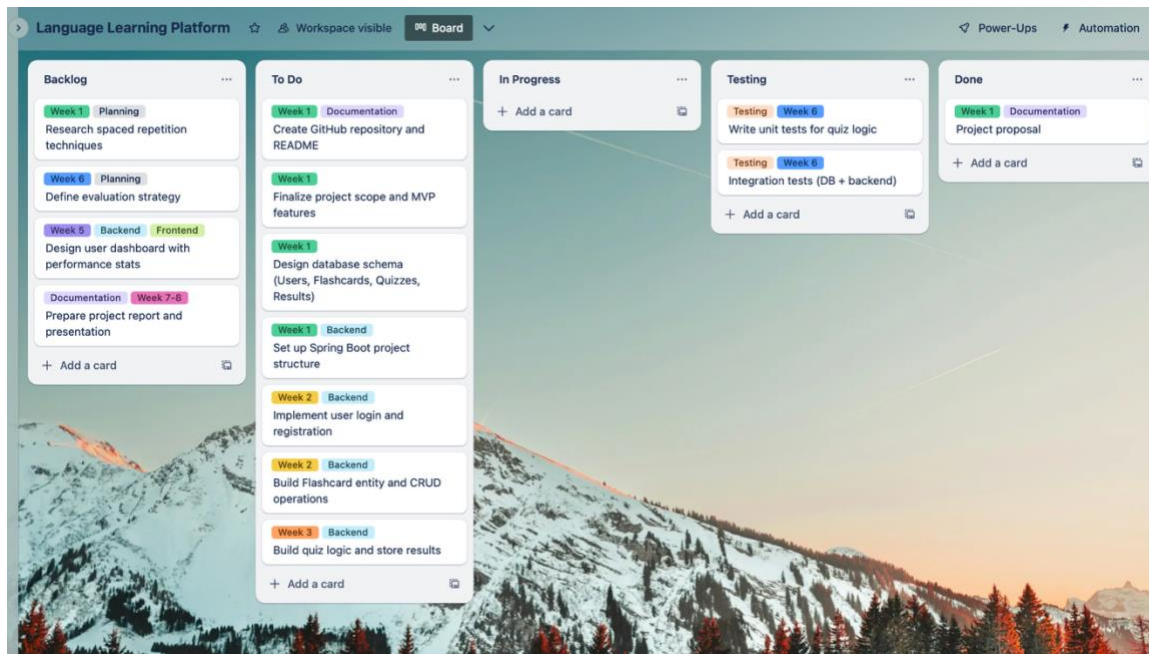


Figure 1: Trello board built to track project development

#### 3.2 Prior Academic Work

This final project expands upon several components developed in previous modules. These prior works serve as a solid foundation for system design, backend logic, and database modeling.

##### 1. OOSE – Project Part 1 (Class Design)

A class diagram was developed to represent core components of the platform, including students, flashcards, practice sessions, and performance tracking. This model provides the object-oriented structure upon which the current implementation will be based. It will be used as the basis for implementing entities and services in Java Spring Boot.

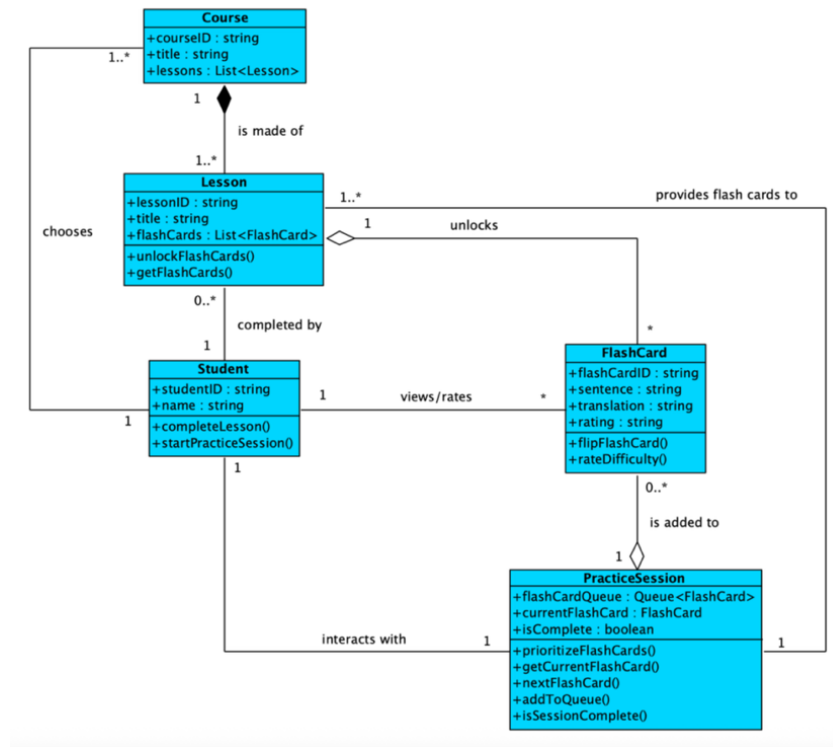


Figure 2: Class diagram from OOSE Project Part 1

## 2. OOSE – Project Part 2 (Agile Implementation + Unit Testing)

The “Practice Sentences” use case was implemented following Agile principles. The tasks were broken down into sprint stories, tracked using Trello, and implemented using a Test-Driven Development (TDD) approach.

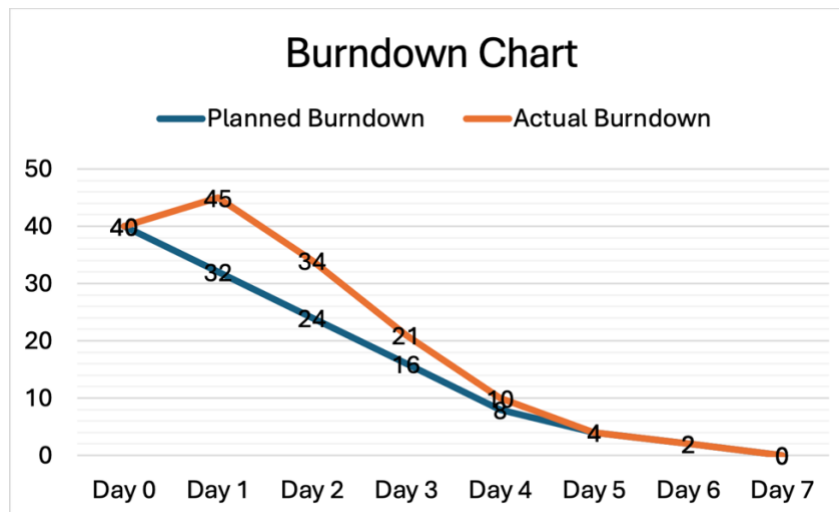


Figure 3: Sprint burndown chart - Demonstrating Agile tracking of progress

Test cases covered: addFlashCard(), flip(), endSession(),  
rateFlashCard().

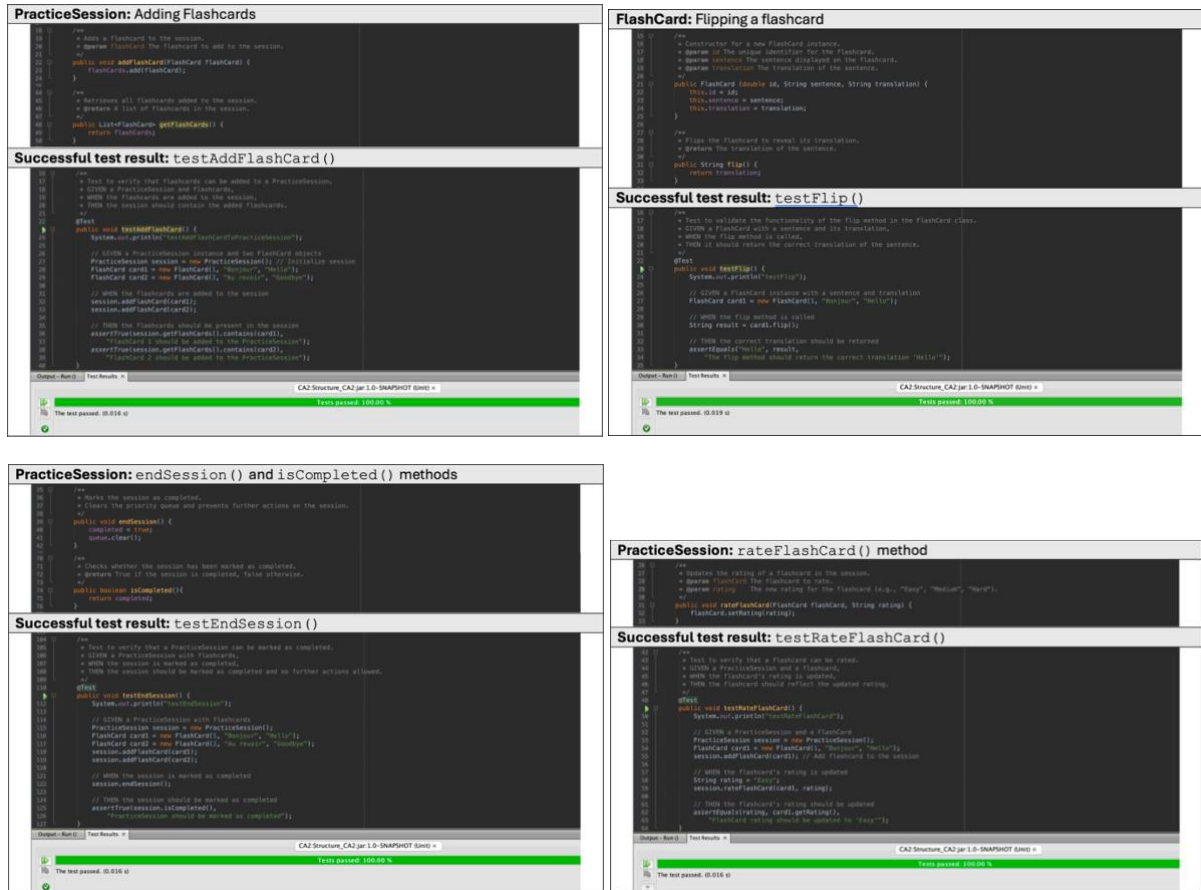


Figure 4: Unit test outputs for flashcard methods – JUnit test output verifying functionality implemented using TDD.

### 3. Databases – CA1 (Database Design and Queries)

The platform's database was carefully designed and normalized. The core schema already includes relationships between users, sessions and flashcards. This will be directly reused and extended in the current system.

ER Diagram

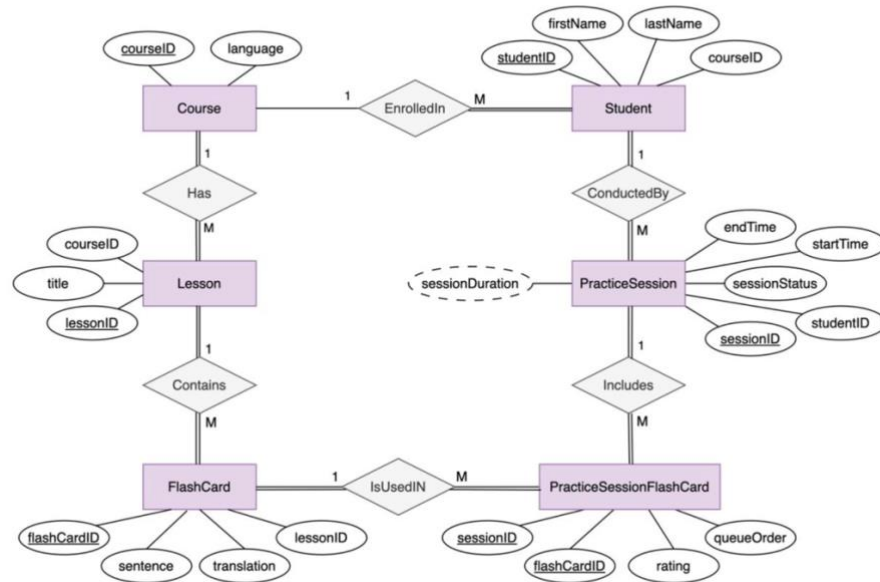


Figure 5: ER Diagram - Shows the relationship between classes

Relational Model

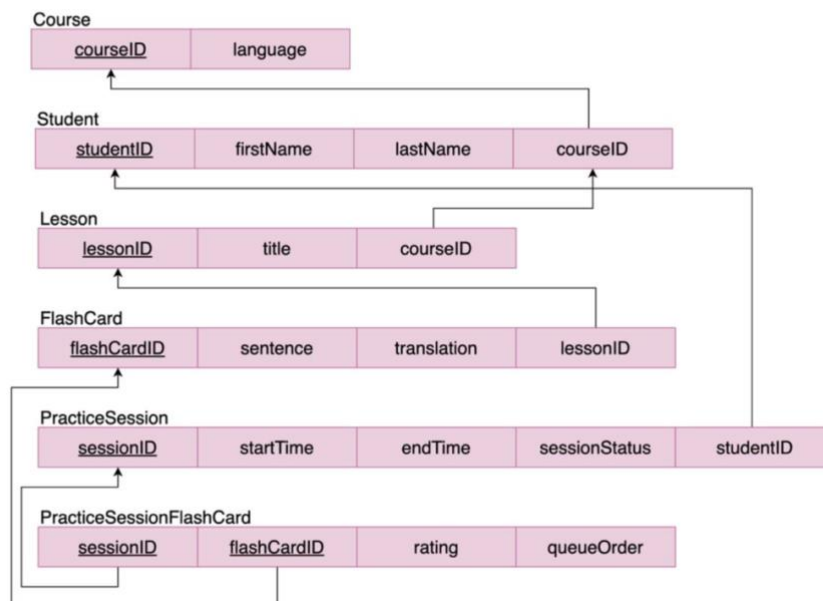


Figure 6: Relational Model - Shows the correspondence between attributes in each table

## 4. Remaining Deliverables

Although some foundational design work has been completed, including object modeling, database design, and initial testing practices, the following components are planned but have not yet been implemented. These represent the main development and testing tasks for the rest of the project timeline.

### 4.1 Features to Be Developed

- **User Authentication:** Implementation of user registration and login using Spring Security, with password hashing and session control.
- **Flashcard CRUD Functionality:** Backend and frontend support for adding, reviewing, and storing private flashcards.
- **Quiz System with Difficulty Selection:** A quiz module that selects flashcards based on difficulty level and spaced repetition logic.
- **Rating and Repetition Logic:** An adaptive system that prioritizes difficult cards based on user ratings (e.g., Easy, Medium, Hard, Don't Know).
- **Progress Dashboard:** A visual overview of user stats such as accuracy, most difficult cards, and session history.
- **Responsive Frontend:** HTML/CSS/Bootstrap-based interface compatible with desktops and mobile services.

### 4.2 Technical Tasks

- **Wireframes and Frontend Layout:** Simple visual mockups for the main screens to guide frontend development.
- **Database Integration:** Migrate the schema from Databases CA1 to MySQL, connect it with Spring Boot using JPA/Hibernate.
- **Spring Boot Setup:** Build out controllers, services, and repositories using the MVC architecture.
- **Testing Framework:** Create new unit tests for upcoming modules (authentication, quiz logic, analytics) and prepare for system/integration testing.

### 4.3 Documentation and Submission

- **Requirements Specification:** A detailed breakdown of system requirements and interface designs.
- **Final Report:** A complete technical report with implementation details, testing results, and evaluation.
- **Final Codebase:** Complete, well-commented Java + SQL code with build instructions.
- **Final Presentation Video:** A video walkthrough explaining the application, features, and development process.



## 5. Main Problems Encountered

- **Adapting previous work:** While earlier modules provided useful foundations (e.g., class diagrams and ER models), adapting these artifacts into a fully integrated web-based system with personalized quizzes, session management, and analytics has introduced new architectural challenges.
- **Tool familiarity:** Although the backend will be developed in Java using Spring Boot, integrating session-based authentication, setting up security layers, and handling user roles involve tools (e.g., Spring Security, Bootstrap components) that require additional learning.

## 6. Expected Stumbling Blocks

- **Spaced repetition algorithm complexity:** Translating the spaced repetition logic from a prototype into a fully persistent, user-adaptive system will involve managing quiz state, ratings, scheduling, and database updates dynamically.
- **Testing and debugging logic-heavy components:** As the quiz system grows in complexity, debugging logic such as flashcard prioritization and user progress analytics may become time-consuming. Ensuring consistency between fronted actions and backend updates will be essential.
- **Time constraints for polishing the frontend:** While the focus is on functionality, some effort must still be allocated to designing a responsive and intuitive interface. This must be balanced against backend priorities like authentication and quiz flow.
- **Integration of modules:** As individual features are built (authentication, quiz engine, rating system, dashboard), ensuring seamless integration and consistent data flow may be difficult – especially when merging backend logic with frontend inputs and user state.

## 7. Focus on Rubric

### 7.1 Communication

So far, I have kept communication clear and consistent – both in written documents and in the organization of my project planning. I have used Trello to visualize the progress and structure of my tasks, which makes it easier to track goals and adjustments.

### 7.2 Writing & Presentation

The project proposal and this interim report were carefully written, using the provided templates and clear formatting. I have included diagrams, screenshots, and structured sections to help convey both progress and planning effectively.

### 7.3 Complexity / Coding

The project involves backend logic for user authentication, spaced repetition algorithms, and personalized quiz delivery – all using Java with Spring Boot. These areas go beyond basic CRUD operations and require logical control over time-based behavior and user tracking.

## **7.4 Innovation**

The platform is based on a real challenge I experienced as an English teacher – helping students build consistent study habits. By using adaptive repetition and progress-based quizzes, the tool goes beyond simple flashcards and provides a tailored learning experience.

## **7.5 Technology**

I am using modern and industry-relevant tools: Spring Boot (Java), MySQL for persistent storage, and a lightweight responsive frontend using Bootstrap. These choices allow me to balance functionality with performance and accessibility.

## **7.6 Completeness**

I have reused and adapted several parts of previous academic work (e.g., ER diagrams, class diagrams, TDD-tested features), which gives me a head start. My focus now is on delivering a fully functional MVP, including authentication, quiz flow, flashcard rating and analytics.

## **7.7 Testing & Evaluation**

Earlier stages of the project used test-driven development (TDD) to validate class behavior, and I plan to extend this with Junit for the new features. Once the main flow is implemented, I also plan to test integration between modules and basic UI functionality.

## **7.8 Project Management**

A Trello board has been set up with columns for each development stage (Backlog, To Do, In Progress, Testing, Done) and labeled tasks by week. I have also planned sprint goals to manage and avoid scope creep. This has already helped me to stay on track and focused.

## **8. Summary of Project Progression**

At this point in the project, I have built the foundation to move forward with development. The original idea was refined based on feedback and personal experience, and the project proposal was completed and approved. I have already completed important early-stage tasks such as outlining the MVP, designing the database and class structures, and organizing development using a Trello board.

In addition, I have reused and adapted previous work – including diagrams, TDD-tested methods, and the core logic for flashcard practice – which gives me a head start in this new version of the system. The goal now is to expand and improve upon these earlier components by integrating user authentication, personalized quizzes, spaced repetition logic, and progress tracking.

Although the coding phase has not fully started, the planning has been solid. I have a clear structure, well-defined deliverables, and a strong idea of the technologies and features I want to include. My focus in the coming weeks will be on completing the backend logic and gradually building a responsive frontend interface, while following agile principles and continuously testing each part.

Overall, I feel well prepared to continue development and confident that the project will evolve into a practical and meaningful application.

# Requirements Specification (RS)

Language Learning Platform

Guilherme Silveira  
6/28/25

# Table of Contents

|        |                                                    |    |
|--------|----------------------------------------------------|----|
| 1      | Introduction                                       | 14 |
| 1.1    | Purpose                                            | 14 |
| 1.2    | Project Scope                                      | 14 |
| 1.3    | Definitions, Acronyms, and Abbreviations           | 14 |
| 2      | User Requirements Definition                       | 14 |
| 3      | Requirements Specification                         | 15 |
| 3.1    | Functional requirements                            | 15 |
| 3.1.1  | Use Case Diagram                                   | 15 |
| 3.1.2  | Requirement 1 <name of requirement in a few words> | 15 |
| 3.1.3  | Requirement 2 <name of requirement in a few words> | 16 |
| 3.2    | Non-Functional Requirements                        | 20 |
| 3.2.1  | Performance/Response time requirement              | 20 |
| 3.2.2  | Availability requirement                           | 20 |
| 3.2.3  | Recover requirement                                | 8  |
| 3.2.4  | Robustness requirement                             | 8  |
| 3.2.5  | Security requirement                               | 20 |
| 3.2.6  | Reliability requirement                            | 8  |
| 3.2.7  | Maintainability requirement                        | 21 |
| 3.2.8  | Portability requirement                            | 8  |
| 3.2.9  | Extendibility requirement                          | 8  |
| 3.2.10 | Reusability requirement                            | 21 |
| 3.2.11 | Resource utilization requirement                   | 8  |
| 4      | GUI                                                | 21 |
| 5      | System Architecture                                | 8  |
| 6      | System evolution                                   | 22 |

# Introduction

## Purpose

The purpose of this project is to develop a language-learning web application that helps users practice vocabulary and sentences using quizzes based on flashcards and spaced repetition techniques. The system is designed to adapt to each learner's progress by adjusting the frequency of flashcards according to the user's performance. The platform aims to provide a practical, personalized, and consistent learning experience.

## Project Scope

This project will deliver a fully functional MVP (Minimum Viable Product) that allows registered users to log in, access private decks of flashcards, complete quizzes with varying difficulty levels, and receive feedback and performance analytics. The backend will be built with Java using Spring Boot, with persistent data storage in MySQL. The frontend will be simple and responsive, implemented using HTML, CSS, and Bootstrap.

## Definitions, Acronyms, and Abbreviations

**MVP:** Minimum Viable Product

**CRUD:** Create, Read, Update, Delete

**UI:** User Interface

**UX:** User Experience

**API:** Application Programming Interface

**MySQL:** A relational database management system

**Spring Boot:** A Java-based framework for backend development

**Spaced Repetition:** A learning technique that increases intervals of time between reviews of previously learned material

## User Requirements Definition

The system is intended for individual language learners who want to improve their vocabulary and language structure through regular practice. Users need a platform that adapts to their learning pace, tracks their progress, and makes studying more consistent and engaging.

The main user requirements are:

- The user must be able to create an account and log in securely.
- The user should be able to select a deck and start a quiz.
- The quiz should present flashcards based on difficulty and user history.
- After answering, the user should be able to rate how easy or hard the flashcard was.
- The system should update the flashcard's repetition frequency based on the user's rating.
- Users should be able to see a summary of their performance after each quiz.

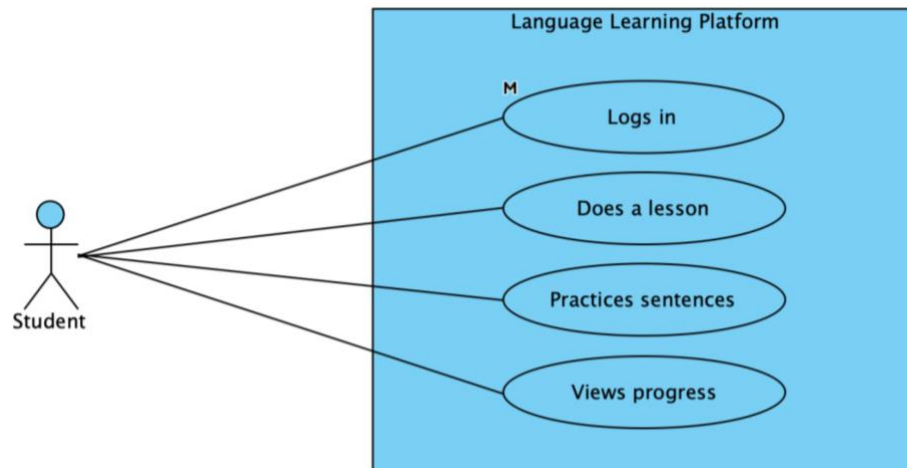
These requirements are meant to ensure that the application is practical, user-friendly, and effective for independent language learning.

## Requirements Specification

### Functional requirements

#### Use Case Diagram

The Use Case Diagram provides an overview of all functional requirements.



#### Requirement 1 – Practice Flashcards

##### *Description & Priority*

This requirement allows users to practice flashcards from a selected deck. Each flashcard is displayed one at a time, allowing the user to flip it to view its translation. The user then rates their familiarity with the flashcard. These ratings are recorded and used to update the order of appearance based on user performance, supporting the spaced repetition method.

**Priority:** High

## Use Case

**Scope:** Flashcard Quiz Functionality

**Description:** This use case describes the process in which a logged-in user engages in a flashcard quiz session using a selected deck.

**Primary Actor:** Registered User (Student)

**Preconditions:**

- User must be logged in.
- At least one deck of flashcards must be available and selected.

**Main flow**

1. System presents the first flashcard.
2. User flips the flashcard to see the translation.
3. User selects a rating: Easy, Medium, Hard, or Don't Know.
4. System records the rating and reorders the deck dynamically.
5. Steps 1-4 repeat until all flashcards are reviewed or session ends.
6. System displays a summary of the user's performance.

**Alternate flow**

A1: If the user decides to skip a flashcard

1. System records it as "Skipped" and presents the next flashcard

**Exceptional flow**

E1: If no flashcards exist in the selected deck

1. System shows an error message and redirects the user to deck selection.

**Post conditions**

- User ratings are stored and linked to the corresponding flashcards.
- Flashcard priority and repetition logic are updated based on user feedback.
- Session summary is stored for analytics.

## Requirement 2 – Rate Flashcards and Adjust Repetition Frequency

### *Description & Priority*

The system must allow users to rate each flashcard based on their understanding (Easy, Medium, Hard, or Don't Know). These ratings will influence how frequently each card appears in future sessions. Cards marked as difficult or unknown will appear more often, while easier ones will be shown less frequently over time, reinforcing the spaced repetition model.

**Priority:** High



## *Use Case*

**Scope:** Adaptive Learning Functionality

**Description:** This use case outlines how flashcard ratings are used to adjust the scheduling of future reviews.

**Primary Actor:** Registered User (Student)

**Preconditions:**

- User must be logged in.
- The flashcard must have been flipped (reviewed).

**Main flow**

1. The user selects a rating (Easy, Medium, Hard, Don't Know).
2. The system records the rating and uses it to update the card's priority in the learning queue.
3. The session continues with the next flashcard.

**Alternate flow**

A1: If the user closes session early:

1. Ratings for already-reviewed flashcards are saved.
2. The session can be resumed later from the last reviewed card.

**Exceptional flow**

E1: If a card fails to load or is incomplete:

1. System skips the card and logs an error.
2. The user is notified and taken to the next card.

**Post conditions:**

- Rating is saved and associated with the specific flashcard and session.
- Card scheduling logic is updated in the backend.

## **Requirement 3 – User Authentication**

### *Description & Priority*

Users must be able to register and log in securely to the system. Only authenticated users will be able to access their personalized flashcard decks, practice history, and performance analytics. This helps ensure data privacy and enables user-specific learning progress tracking.

**Priority:** High

## *Use Case*

**Scope:** Account Management

**Description:** This use case allows a user to register for an account or log in to access personalized features.

**Primary Actor:** Student

**Preconditions:**

- For login: Account must already exist.
- For registration: Valid email and password must be provided.

**Main flow - Registration**

1. User clicks “Register” on the homepage.
2. User enters email and password.
3. System validates input and creates the account.
4. System logs the user in and redirects them to the dashboard.

**Main flow - Login**

1. User clicks “Login” on the homepage.
2. User enters email and password.
3. System verifies credentials.
4. If correct, user is logged in and redirected to the dashboard.

**Alternate flow**

A1: If the user enters incorrect password:

1. System displays an error message and prompts to try again.

**Exceptional flow**

E1: If email is not registered:

1. System informs the user and suggests account creation.

**Post conditions:**

- Authenticated session is started.
- User is redirected to the dashboard.

## Requirement 4 – View Performance Statistics

### *Description & Priority*

The system should display personalized performance analytics to help students understand their learning progress. This includes statistics such as quiz accuracy, flashcard difficulty trends, most missed items, and overall improvement over time.

**Priority:** Medium

### *Use Case*

**Scope:** Analytics and Feedback

**Description:** This use case allows students to visualize their progress and identify areas for improvement.

**Primary Actor:** Student

**Preconditions:**

- User must be logged in.
- The user must have completed at least one quiz or flashcard session.

**Main flow**

1. User logs in and access the dashboard.
2. User clicks “My Performance”.
3. System retrieves historical data related to quiz and flashcard activity.
4. System displays charts and indicators showing accuracy rates, difficulty breakdown, and progress trends.

**Alternate flow**

A1: If no activity is recorded yet:

1. System shows a message like “Start your first session to see progress here”.

**Exceptional flow**

E1: If data retrieval fails due to server error:

1. System displays a message and suggests retrying later.

**Post conditions:**

- The system displays performance statistics retrieved from the database.

## Requirement 5 – Manage Flashcard Decks

### *Description & Priority*

Only authorized admins or system managers should be able to create, edit, or delete flashcard decks and the flashcards within them. This ensures content quality and prevents unauthorized changes to educational material. Students will only be able to view and interact with the decks assigned to them.

**Priority:** Medium

### *Use Case*

**Scope:** Content Management

**Description:** This use case describes how an admin manages the flashcard decks and their content.

**Primary Actor:** Admin

**Preconditions:**

- Admin must be logged in.

**Main flow – Create Deck**

1. Admin logs in and access the deck management page.
2. Admin clicks “Create New Deck”.
3. Admin provides a name and category.
4. System stores the new deck.

**Main flow – Add/Edit/Delete Flashcards**

1. Admin selects a deck.
2. Admin clicks to add, edit, or delete a flashcard.
3. Admin fills in the sentence and translation fields as needed.
4. System saves the changes and updates the deck contents.

**Alternate flow**

A1: Admin cancels the operation:

1. System discards changes and returns to the deck overview.

**Exceptional flow**

E1: If fields are invalid (e.g., empty or duplicate):

1. System displays an error and prevents saving.

**Post conditions:**

- Changes to decks or flashcards are reflected in the system and available to students for practice.

## Non-Functional Requirements

### Performance Requirements

The system should deliver responses to user actions, such as loading quizzes or submitting answers, within 2 seconds under normal network conditions. The backend should support at least 50 concurrent users without significant performance degradation.

### Availability requirement

The system should be available at least 95% of the time, excluding maintenance periods. In case of server downtime, users should be shown a friendly maintenance message.

### Security Requirements

User data must be protected through secure authentication mechanisms and encrypted password storage. Only authenticated users can access practice history and personal analytics. Admin operations (e.g., managing decks) must be restricted to authorized roles.

## Maintainability requirement

The system codebase should follow modular and well-documented practices, allowing future contributors to add features or fix bugs with minimal ramp-up time. Use of standard frameworks like Spring Boot and RESTful APIs ensures maintainability.

## Usability Requirement

The interface should be intuitive and accessible to language learners of different levels. A responsive layout is required so users can access the system comfortably from desktops, tablets, or smartphones.

# Interface requirements

## User Interfaces

The system will provide a clean, responsive web interface designed primarily for students. Key interface elements will include:

**Login/Register page:** Secure authentication for students and admins.

**Dashboard:** Overview of recent activity, available quizzes, and performance summary.

**Quiz page:** Interactive layout to answer sentence-based flashcards with difficulty rating.

**Performance page:** Displays user statistics such as accuracy and progress over time.

**Admin panel (restricted):** For creating, editing, and deleting decks and flashcards.

The UI will be developed using HTML, CSS, and optionally Bootstrap, and rendered via Spring Boot using simple templating or server-side rendering.

## Hardware Interfaces

No special hardware is required. The application will be accessible via modern web browsers on common devices such as:

- Desktop computers (Windows, macOS, Linux)
- Tablets and smartphones (iOS, Android)

## Software Interfaces

**Frontend:** HTML/CSS/Bootstrap

**Backend:** Java with Spring Boot (RESTful API)

**Database:** MySQL for persistent storage of user data, quizzes, flashcards, and results

**Development Tools:** Visual Studio Code, Visual Paradigm, MySQL Workbench, Trello

**Security:** Spring Security (planned for role-based access and login authentication)

## Communication Interfaces

The system will follow the client-server model:

- HTTP/HTTPS requests between the client interface and the Spring Boot backend
- JSON format for communication between frontend and backend
- JDBC or Spring Data for communication between the backend and MySQL database

## System Evolution

This system could allow future enhancements without major structural changes. Possible improvements include adding teacher accounts to manage students, implementing gamification features to increase engagement, supporting audio-based exercises for listening and pronunciation, and eventually developing a mobile app. These additions aim to make the platform more interactive, accessible, and adaptable to different learning needs.